

PROJECT 02

ModelSentinel: An Uncertainty-Aware Multi-Agent Forensic Framework for AI Model Supply-Chain Security

1. Problem Statement

Organizations increasingly obtain pre-trained AI models from public repositories, vendors, research partners, and internal development teams. These files may contain millions or billions of numerical parameters and are often trusted because they use a recognized format or continue to deliver acceptable model performance.

However, a model file can be structurally valid while containing suspicious data hidden within its numerical weights. Low-order bytes or selected tensor regions can potentially be modified to store hidden patterns without significantly affecting the model's intended functionality. Such modifications may remain undetected during normal file validation or functional testing.

Conventional security solutions address related risks but do not fully inspect the numerical contents of model weights:

- Antivirus tools primarily detect known malicious signatures.
- Pickle scanners identify unsafe serialization instructions.
- Digital signatures confirm origin and file integrity but cannot establish that the original content was benign.
- Behavioural testing requires the model to be loaded and executed.
- SafeTensors reduces arbitrary-code deserialization risks but does not confirm that tensor values are free from hidden or abnormal content.

This creates an AI supply-chain security gap. A suspicious model artifact could enter model registries, development pipelines, cloud workloads, customer applications, or regulated environments before existing controls identify the risk.

The potential impact includes security incidents, investigation and recovery costs, operational disruption, regulatory concerns, and reduced customer confidence in externally sourced AI models.

ModelMRI addresses this gap by providing a non-executing, explainable security scanner that examines model-weight content before the model is trusted or deployed.

2. Objectives in Detail

Objective 1: Inspect models without executing untrusted code

ModelMRI will analyze supported model files without importing model source code, loading unsafe serialization logic, or performing inference. Strict validation and bounded reading will reduce the security and resource risks associated with processing untrusted artifacts.

Objective 2: Detect suspicious patterns inside model weights

The system will divide tensors into manageable blocks and analyze numerical and byte-level characteristics. It will identify unusual entropy, compression, repetition, printable-byte patterns, dominant-byte behaviour, and statistical deviations that may indicate hidden or manipulated content.

Objective 3: Localize suspicious content precisely

Instead of producing only a file-level alert, ModelMRI will identify the tensor name, block number, element range, and probability associated with the highest-risk region. This will allow security analysts to investigate a specific location rather than manually reviewing an entire model.

Objective 4: Provide explainable security decisions

Every result will include the features that influenced the prediction. The system will produce transparent evidence supporting a CLEAN, REVIEW, or SUSPICIOUS classification and an operational ALLOW, REVIEW, or QUARANTINE recommendation.

Objective 5: Support enterprise security workflows

ModelMRI will generate machine-readable JSON and human-readable HTML reports. These outputs can support integration with model registries, CI/CD pipelines, security operations platforms, audit processes, and model marketplaces.

Objective 6: Establish a safe and reproducible evaluation framework

The project will use harmless canaries and synthetic patterns instead of real malware. Dataset generation, splitting, feature extraction, training, and evaluation will be deterministic and reproducible.

Objective 7: Prepare the solution for broader model ecosystems

Future development will extend the scanner from SafeTensors and FP32 data to GGUF, FP16, BF16, INT8, INT4, adapters, and sharded model files. Evaluation will also cover unseen model families and previously unseen hiding techniques.

3. Dataset: Generation, Collection and Relevance

The current prototype uses a deterministic synthetic dataset generated locally with Python and NumPy. Synthetic generation was selected because verified real-world model-weight malware datasets are limited, difficult to distribute legally, and potentially unsafe.

The dataset contains clean SafeTensors model artifacts and corresponding suspicious versions. Suspicious examples are produced by embedding harmless repeated canary text into selected low-order bytes of specific tensor blocks. The canaries are non-executable research markers, such as “MODELMRI_SAFE_CANARY” and “NON_EXECUTABLE_AUDIT.” No operational malware is created, stored, extracted, or executed.

The dataset contains:

- 20 model artifacts.
- 580 tensor-block records.
- 10 labelled positive blocks.
- A positive-block rate of approximately 1.724%.
- Blocks containing 4,096 tensor values.
- 25 numerical and byte-level features per block.

The dataset is divided as follows:

- Training: 12 artifacts, 348 blocks and 6 positive blocks.
- Validation: 4 artifacts, 116 blocks and 2 positive blocks.
- Testing: 4 artifacts, 116 blocks and 2 positive blocks.

Every record contains an artifact-level label and a block-level label. The block-level ground truth identifies the precise tensor and block containing the harmless embedded pattern. Clean and suspicious versions of the same base artifact remain in the same split to reduce data leakage.

Dataset relevance to the objectives:

- Block labels validate the localization objective.
- Clean and suspicious pairs support controlled feature comparison.
- Class imbalance tests the class-weighted detection approach.
- Harmless canaries support safe and repeatable experimentation.

- Train, validation, and test separation supports threshold selection and unbiased prototype evaluation.
- SafeTensors artifacts validate non-executing parsing and analysis.

The dataset is appropriate for demonstrating pipeline correctness. However, it does not represent the full diversity of real-world models or attacks. Production validation will require legitimate models from different families, precisions, quantization methods, fine-tuning processes, and benign watermarking systems.

4. Technical Stack

Programming and Data Processing

- Python 3.11+
- NumPy for tensor and numerical operations
- pandas for dataset preparation and analysis
- Pydantic for schema and input validation
- hashlib for artifact hashing
- zlib for compression-related features
- JSON processing for metadata and model parameters

Model and Data Formats

- SafeTensors for non-executable model-weight storage
- CSV and Parquet for feature datasets
- JSON for predictions, metrics and model parameters
- HTML for human-readable security reports

Feature Engineering

ModelMRI extracts 25 features covering:

- Mean, standard deviation and root mean square
- Skewness and kurtosis
- Percentile and interquartile ranges
- Zero and near-zero fractions
- Unique-value ratio
- Lag-one autocorrelation

- Raw-byte entropy and compression
- Low-byte entropy and compression
- Printable-byte and repetition ratios
- Dominant-byte fraction
- Bit-balance measurements
- Composite structured low-byte score

Detection Models

- Robust statistical anomaly detection using median and Median Absolute Deviation
- Class-weighted L2 logistic regression
- Training-only feature standardization
- Validation-based decision-threshold selection
- Block-level probability scoring
- Artifact-level aggregation using the highest-risk block
- Feature-contribution explanations

Interfaces and Quality Assurance

- Command-line interface
- Streamlit dashboard
- Altair visualizations
- JSON and HTML reporting
- pytest automated testing
- Deterministic dataset generation

Planned Production Components

- FastAPI service
- Docker-based isolated scanning workers
- Model-registry webhooks
- SIEM integration
- PostgreSQL and object storage

- Signed evidence and audit records
- GGUF and quantized-model support

5. Expected Outcomes

ModelMRI is expected to deliver the following outcomes:

- A functional pre-deployment scanner for supported AI model artifacts.
- Safe structural validation without executing untrusted model code.
- Automated detection of suspicious statistical and byte-level patterns.
- Accurate identification of the highest-risk tensor and block.
- Explainable risk scores and feature-level evidence.
- CLEAN, REVIEW, or SUSPICIOUS classifications.
- ALLOW, REVIEW, or QUARANTINE policy recommendations.
- JSON and HTML reports suitable for audit and enterprise integration.
- A reproducible benchmark for model-weight security research.
- Reduced manual investigation effort for security and MLOps teams.
- Improved trust in externally sourced AI models.
- A foundation for commercial API, registry, marketplace, and on-premises scanning solutions.